

OTIMIZAÇÃO DE CARREGAMENTO DE CONTÊINERES EM BALSAS

Abordagem com Algoritmos Genéticos para Maximizar Estabilidade e Minimizar Movimentos

Equipe 2

- Antonio Gabriel Nunes Martins
- Carlos Eduardo Correa Queiroz
- Cristiano Peniche Ceccon
- Eduarda Aline de Oliveira Noronha
- João Victor Mattos Lopes
- Leonardo Fernandes Cavalcante
- Lucas dos Anjos Silva
- Paulo Eduardo Ribeiro Cativo
- Pedro César Mendonça Ituassú
- Thiago Cordeiro de Melo



O QUE SÃO ALGORITMOS GENÉTICOS (AG)?

- Inspirados na evolução natural (seleção, crossover, mutação).
- Úteis para problemas de otimização complexos (**como o carregamento de contêineres**).

NOSSO PROBLEMA

- Carregar **24 contêineres** em uma **balsa 4x4** (máx. 10/andar e min. 8/andar), minimizando movimentos e maximizando estabilidade.
- **Restrições:** Contêineres inicialmente empilhados (12/andar no porto), mesmo peso/dimensão.
- A balsa deve também **afundar** com uma determinada distribuição dos contêineres



ALGORITMO GENÉTICO ADAPTATIVO (AGA)

- Diferença para AG clássico:
 - Taxas de mutação e crossover dinâmicas (ajustam-se conforme desempenho da população).
 - Exemplo: Se diversidade genética é baixa, aumenta mutação para evitar estagnação.
- Por que usamos?
 - Exploração para evitar estagnação e melhorar a busca por soluções ótimas.



FLUXO RESUMIDO DO ALGORITMO

1. Definição das variáveis iniciais:

- Parâmetros clássicos (população, gerações, mutação, crossover).
- Taxas de mutação/crossover mínimas e máximas para controle adaptativo.

2. Inicialização (run_aga):

- Geração aleatória dos cromossomos $[[X1, Y1, Z1], [X2, Y2, Z2]]$ para 24 contêineres.
- Verificação de suporte (sem contêineres flutuando) via `selecionarPosicao`.

3. Avaliação da População:

- Cálculo da função objetivo (estabilidade + quantidade de movimentos).
- Aplicação de penalidades (formato cruz, limites por deck, movimentos excessivos).
- $\text{Fitness} = 1 / (1 + \text{função objetivo})$.

4. Seleção:

- Método da roleta viciada (probabilístico, favorecendo melhores indivíduos), baseando-se na somatória de todas as avaliações dos indivíduos de uma dada geração.

5. Crossover (One-Point):

- Corte aleatório nos cromossomos e recombinação dos genes.
- Aplicação de função de reparo para manter soluções válidas.

6. Mutação:

- Swap das posições finais de dois contêineres.
- Recalcular movimentos.

7. Construção das Novas Gerações:

- Repetir seleção, crossover e mutação até formar nova população.

8. Evolução:

- Atualizar histórico de melhores indivíduos (por geração e globalmente).
- Ajuste dinâmico das taxas de mutação e crossover de acordo com a estagnação das gerações.

9. Finalização:

- Plotagem 2D/3D da melhor solução encontrada (posição dos contêineres na balsa).
- Plotagem da evolução do Algoritmo Genético ao longo do processamento através de um Gráfico de 3 Eixos.

FLUXO DO CÓDIGO

```
def construirCromossomo(self):
    cromossomo = []
    # Construir as posições iniciais dos Contêineres.
    posicoesIniciaisPossiveis = []
    # Montar posições iniciais possíveis
    for lines in range(self.matrizTerminal[0]):
        for columns in range(self.matrizTerminal[1]):
            for decks in range(self.matrizTerminal[2]):
                posicoesIniciaisPossiveis.append([lines + 1, columns + 1, decks + 1])
    # Montar posições finais possíveis.
    for lines in range(self.matrizBalsa[0]):
        for columns in range(self.matrizBalsa[1]):
            for decks in range(self.matrizBalsa[2]):
                self.posicoesFinaisPossiveis.append([lines + 1, columns + 1, decks + 1])
    for i in range(len(posicoesIniciaisPossiveis)):
        # Escolher aleatoriamente a posição final de cada contêiner já no Terminal Portuário
        selected = self.selecionarPosicao()
        cromossomo.append([posicoesIniciaisPossiveis[i], selected])
    return cromossomo
```

ETAPA 1 DO FLUXO DO CÓDIGO (ESTRUTURAR CROMOSSOMO)

A função acima estrutura o cromossomo do indivíduo da seguinte forma:

1. As posições iniciais e as possíveis posições finais são instanciadas de acordo com as matrizes fornecidas da Balsa, bem como do Terminal Portuário.
2. Para todos os contêineres no terminal portuário (identificados pela sua posição inicial), é preciso se saber uma posição final na balsa, que é escolhida aleatoriamente inicialmente, isto é feito através de uma função `selecionarPosicao()`, que também previne contra o problema de contêineres flutuarem e desobedecerem às restrições.
3. O cromossomo é estruturado como sendo uma “lista de listas”, disposto da seguinte forma:
 - `[[X1, Y1, Z1], [X2, Y2, Z2]]`
 - X1, Y1 e Z1 são as coordenadas iniciais;
 - X2, Y2 e Z2 são as coordenadas finais.

ETAPA 2 DO FLUXO DO CÓDIGO (FUNÇÃO FITNESS)

- Função biobjetiva (minimizar instabilidade e minimizar a quantidade de movimentos)
 - Utilização de pesos (“alfa” e “beta”) para determinar a prioridade da solução (estabilidade ou quantidade de movimentos).
- A função de fitness do problema ficou encarregada de utilizar as informações de um indivíduo para tentar solucionar o problema com os genes dos seus cromossomos.

```
# Função de fitness atualizada com maior ênfase na formação em cruz
def fitness(self, alfa=0.7, beta=0.3):
```

```
# Centro geométrico da balsa
centro_geo = [self.matrizBalsa[0] / 2 + 0.5, self.matrizBalsa[1] / 2 + 0.5, self.matrizBalsa[2] / 2 + 0.5]

# Calcular o centro de massa real
centro_massa = np.mean([gene[1][0], gene[1][1], gene[1][2]] for gene in self.cromossomo], axis=0)
```

- Balanço (Estabilidade)
 - Mede a distância entre o centro de massa real e o ideal (quanto maior, mais instável)

```
# Calcular o desequilíbrio (distância entre centro de massa e centro geométrico)
balanco_x = (centro_massa[0] - centro_geo[0]) ** 2
balanco_y = (centro_massa[1] - centro_geo[1]) ** 2
balanco = np.sqrt(balanco_x + balanco_y) # Distância euclidiana no plano XY
```

- Penalidades:
 - São aplicadas conforme as restrições vão sendo violadas. As restrições (heurísticas) utilizadas foram as que penalizam a disposição ideal que identifica estabilidade, bem como a violação da quantidade por deck e a quantidade de movimentos sendo alta.

```
# Penalidade para distribuição não-cruz (com peso muito maior)
penalidade_cruz = self.calcular_penalidade_cruz(centro_geo) * 3.0 # Triplicar o efeito

# Penalidade para decks com menos que o mínimo (8) ou mais que o máximo (10)
penalidade_deck = self.calcular_penalidade_deck(8, 10)

# Penalidade para movimentos excessivos
if movimentos > 190:
    penalidade += 10
```

FLUXO DO CÓDIGO (FUNÇÃO FITNESS)

- Fórmula Final
 - Quanto maior o fitness (mais próximo de 1), melhor a solução
 - O denominador soma todos os aspectos negativos (balanceamento, penalidades, movimentos)

```
# Fórmula de fitness combinada
fitness = 1 / (1 + (alfa * (balanco + penalidade_cruz)) + (beta * movimentos) + penalidade + penalidade_deck)
```

$$\text{Fitness} = \frac{1}{1 + \alpha \cdot (\text{balanço} + \text{penalidade_cruz}) + \beta \cdot \text{movimentos} + \text{penalidade} + \text{penalidade_deck}}$$

- Componentes da Fórmula:
 - α (0.7): Peso do equilíbrio (balanço + formação em cruz)
 - β (0.3): Peso da eficiência (movimentos)
 - balanço: Distância euclidiana entre centro de massa e centro geométrico
 - penalidade_cruz: Penalização por desvios da formação em cruz ($\times 3 \times 3$)
 - movimentos: Total de movimentos realizados
 - penalidade_deck: Violação de limites de contêineres por deck

$$\text{Fitness} \in (0, 1]$$

OBS: Quanto maior o valor (próximo de 1), melhor a solução. O denominador agrega todas as penalidades.

FLUXO DO CÓDIGO (PENALIDADE)

- Penalidade por Deck
 - Objetivo: Garantir 8-10 contêineres por deck.
 - Lógica: Penalidade quadrática por violação.
- Penalidade por Formação
 - Objetivo: Forçar disposição em cruz.
 - Lógica: Penalidade exponencial por distância do centro.

```
def calcular_penalidade_deck(self, min_deck, max_deck):  
    """  
    Penaliza decks que têm menos que min_deck ou mais que max_deck contêineres  
    """  
    penalidade = 0  
  
    # Contar contêineres por deck  
    contagem_deck = [0] * self.matrizBalsa[2]  
    for gene in self.cromossomo:  
        deck = gene[1][2] - 1 # Ajustar para índice baseado em zero  
        contagem_deck[deck] += 1  
  
    # Penalizar decks fora dos limites  
    for count in contagem_deck:  
        if count < min_deck:  
            penalidade += (min_deck - count) ** 2 # Penalidade quadrática  
        elif count > max_deck:  
            penalidade += (count - max_deck) ** 2 # Penalidade quadrática  
  
    return penalidade * 0.5 # Fator de escala para a penalidade  
  
def calcular_penalidade_cruz(self, centro_geo):  
    """  
    Calcula uma penalidade para formações que não seguem um padrão de cruz.  
    Fortemente favorece contêineres no centro e nas linhas/colunas centrais.  
    """  
    penalidade = 0  
  
    # Definir as linhas e colunas que formam a cruz central  
    linhas_centrais = [int(centro_geo[0]), int(centro_geo[0]) - 1]  
    colunas_centrais = [int(centro_geo[1]), int(centro_geo[1]) - 1]  
  
    # Para cada contêiner, verificar se está na cruz central  
    for gene in self.cromossomo:  
        pos = gene[1]  
        linha, coluna = pos[0], pos[1]  
  
        # Se não estiver na cruz, aplicar penalidade significativa  
        if linha not in linhas_centrais and coluna not in colunas_centrais:  
            # Distância até a linha ou coluna central mais próxima  
            dist_linha = min([abs(linha - l) for l in linhas_centrais])  
            dist_coluna = min([abs(coluna - c) for c in colunas_centrais])  
            dist_min = min(dist_linha, dist_coluna)  
  
            # Penalidade exponencial para distância da cruz  
            penalidade += dist_min ** 2.5 # Exponente maior para penalidade mais severa  
  
    return penalidade / len(self.cromossomo) # Normalizar pela quantidade de contêineres
```

FLUXO DO CÓDIGO (SELECIONAR POSIÇÃO)

- Objetivo
 - Escolhe a melhor posição na balsa para cada contêiner, priorizando:
 - Centro da balsa (formação em cruz)
 - Respeito aos limites por deck (8-10 contêineres)
 - Suporte físico (contêineres não flutuantes)
- Priorização de Posições
 - $posicoes_ordenadas = [centro] + [cruz] + [outras]$ # Ordenadas por proximidade ao centro
- Restrições Verificadas
 - Deck não está cheio ($qntPerDeck < maxPerDeck$)
 - Tem contêiner abaixo (exceto no deck 1)
- Fallback
 - Se não achar posição ideal, usa seleção aleatória com as mesmas restrições.

FLUXO DO CÓDIGO (SELECIONAR POSIÇÃO)

```
# Método selecionarPosicao modificado para garantir uma formação em cruz
def selecionarPosicao(self):
    # Centro geométrico da balsa
    centro_geo = [self.matrizBalsa[0] / 2 + 0.5, self.matrizBalsa[1] / 2 + 0.5, self.matrizBalsa[2] / 2 + 0.5]

    # Definir as linhas e colunas que formam a cruz
    linhas_centrais = [int(centro_geo[0]), int(centro_geo[0]) - 1]
    colunas_centrais = [int(centro_geo[1]), int(centro_geo[1]) - 1]

    # Lista de posições disponíveis
    if not self.posicoesFinaisPossiveis:
        raise Exception("Sem posições disponíveis restantes")

    # Classificar posições em 3 categorias:
    # 1. Posições no centro absoluto (interseção das cruzes)
    # 2. Posições nas linhas/colunas centrais (restante da cruz)
    # 3. Todas as outras posições
    posicoes_centro = []
    posicoes_cruz = []
    posicoes_outras = []

    for pos in self.posicoesFinaisPossiveis:
        linha, coluna = pos[0], pos[1]
        # Centro absoluto (onde as linhas e colunas centrais se cruzam)
        if linha in linhas_centrais and coluna in colunas_centrais:
            posicoes_centro.append(pos)
        # Restante da cruz (linhas ou colunas centrais)
        elif linha in linhas_centrais or coluna in colunas_centrais:
            posicoes_cruz.append(pos)
        # Outras posições
        else:
            posicoes_outras.append(pos)

    # Ordenar cada categoria por proximidade ao centro geométrico
    for lista in [posicoes_centro, posicoes_cruz, posicoes_outras]:
        lista.sort(key=lambda pos: np.sqrt((pos[0] - centro_geo[0]) ** 2 + (pos[1] - centro_geo[1]) ** 2))
```

```
# Tentar selecionar uma posição válida, começando pelas melhores
for i, posicaoSelecionada in enumerate(posicoes_ordenadas):
    z = posicaoSelecionada[2]
    abaixo = [posicaoSelecionada[0], posicaoSelecionada[1], z - 1]

    # Se há contêiner abaixo, então continue para a próxima posição
    # (não tentamos descer para garantir mais controle sobre a seleção)
    if abaixo in self.posicoesFinaisPossiveis:
        continue

    # Verificar restrições de deck
    deck = z
    if self.qntPerDeck[deck - 1] < self.maxPerDeck:
        self.qntPerDeck[deck - 1] += 1
        self.posicoesFinaisPossiveis.remove(posicaoSelecionada)

    # Se chegou no máximo, remove todas posições restantes desse deck
    if self.qntPerDeck[deck - 1] == self.maxPerDeck:
        self.posicoesFinaisPossiveis = [
            pos for pos in self.posicoesFinaisPossiveis if pos[2] != deck
        ]
    return posicaoSelecionada

# Se não encontrou posição válida (improvável com a ordenação acima)
# Use a abordagem original como fallback
while True:
    if not self.posicoesFinaisPossiveis:
        raise Exception("Sem posições disponíveis restantes")

    posicaoSelecionada = self.posicoesFinaisPossiveis[round(random() * (len(self.posicoesFinaisPossiveis) - 1))]
    z = posicaoSelecionada[2]
    abaixo = [posicaoSelecionada[0], posicaoSelecionada[1], z - 1]

    if abaixo in self.posicoesFinaisPossiveis:
        posicaoSelecionada = abaixo
        continue
    else:
        deck = z
        if self.qntPerDeck[deck - 1] < self.maxPerDeck:
            self.qntPerDeck[deck - 1] += 1
            self.posicoesFinaisPossiveis.remove(posicaoSelecionada)

            if self.qntPerDeck[deck - 1] == self.maxPerDeck:
                self.posicoesFinaisPossiveis = [
                    pos for pos in self.posicoesFinaisPossiveis if pos[2] != deck
                ]
            break
        else:
            self.posicoesFinaisPossiveis.remove(posicaoSelecionada)
            continue

return posicaoSelecionada
```

IMPLEMENTAÇÃO DO ALGORITMO GENÉTICO ADAPTATIVO

- Objetivo
 - Estabilidade da balsa (formação em cruz)
 - Minimização de movimentos
 - Restrições de capacidade por deck
- Fluxo Principal (*run_aga*)
 - Inicialização:
 - Cria população inicial de soluções aleatórias
 - Avalia cada solução usando a função de fitness
- Loop de Gerações:

```
for geracao in range(NUM_GER):  
    soma_avaliacao = self.totalAvaliacao()
```

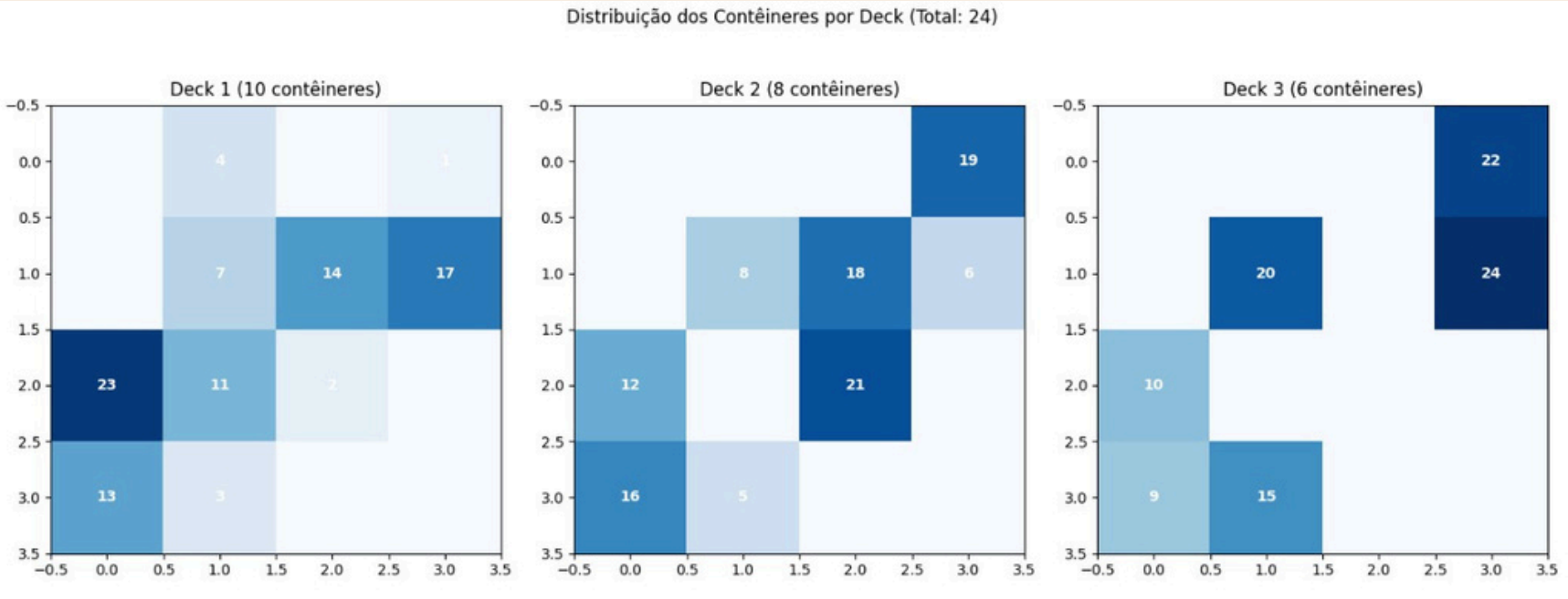
IMPLEMENTAÇÃO DO ALGORITMO GENÉTICO ADAPTATIVO

- Mecanismos-Chave
 - Seleção por Roleta: Dá preferência, mas não exclusividade, às melhores soluções
 - Crossover Adaptativo: Taxa ajustável (70-95%) com reparo de soluções inválidas
 - Mutação Inteligente: Swap que respeita restrições físicas
- Diferenciais Importantes
 - Elitismo: Mantém a melhor solução a cada geração
 - Reparo Automático: Corrige soluções que violam restrições
 - Penalidades:
 - Formação não-cruzada (peso triplo)
 - Decks fora do limite (8-10 contêineres)
 - Movimentos excessivos (> 190)

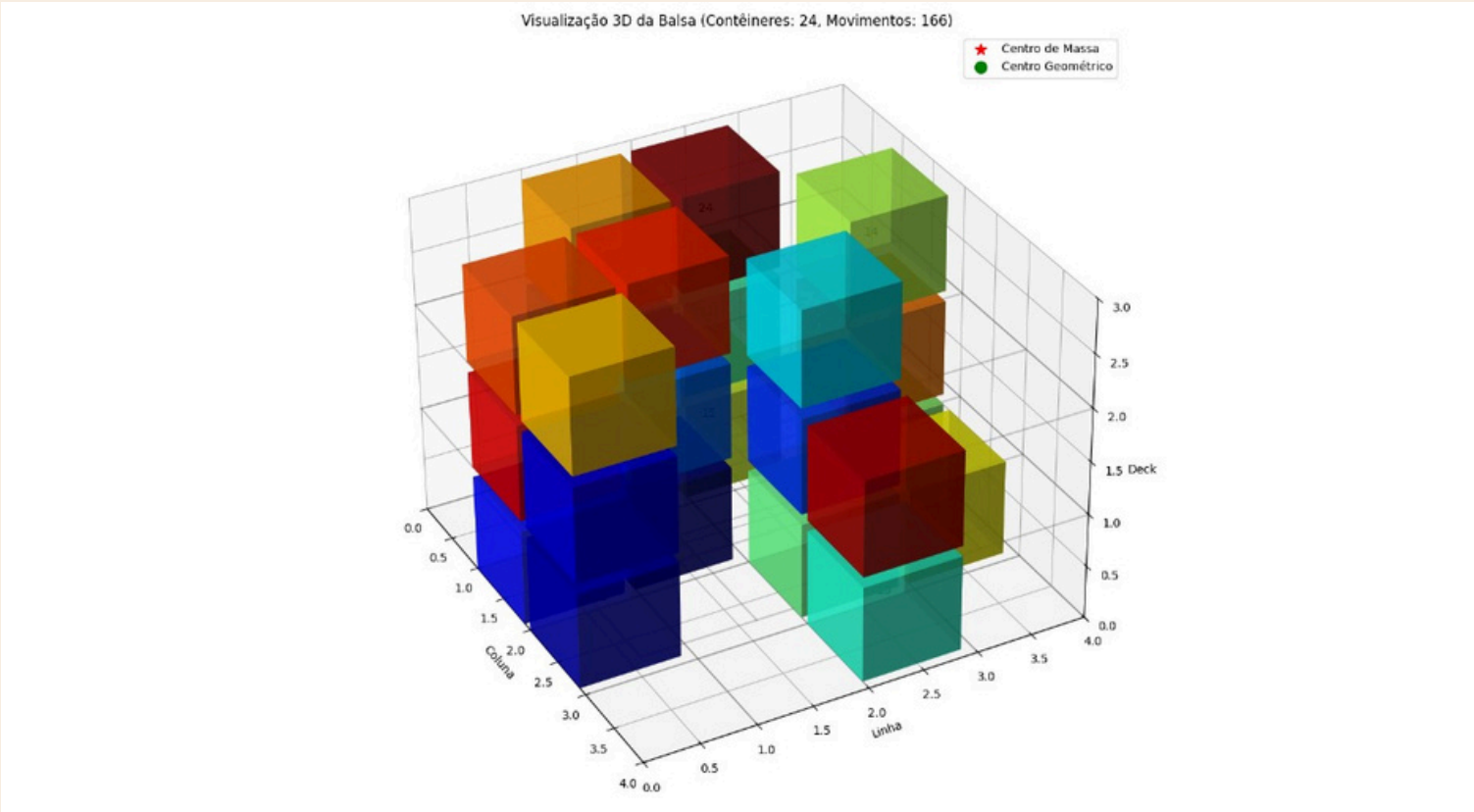
NOSSOS RESULTADOS

Equipe 2

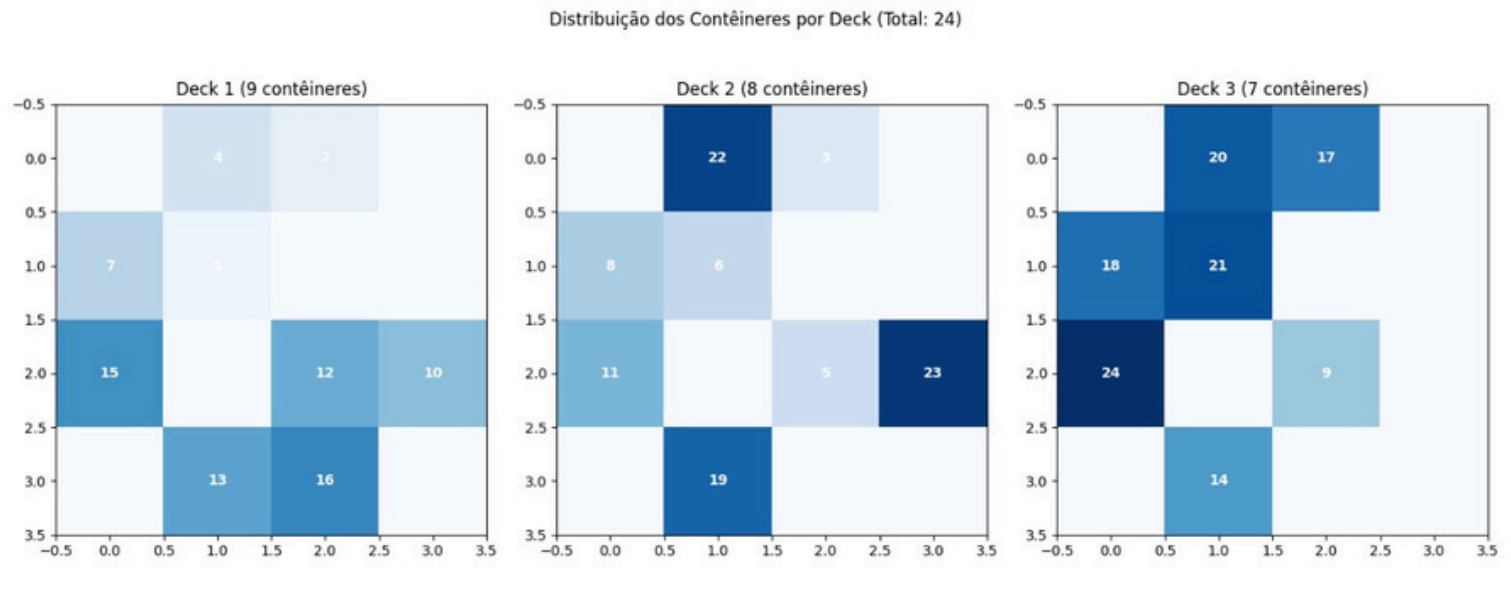
DISTRIBUIÇÃO ENCONTRADA NA ATIVIDADE PRÁTICA



SOLUÇÃO ÓTIMA EM 3D



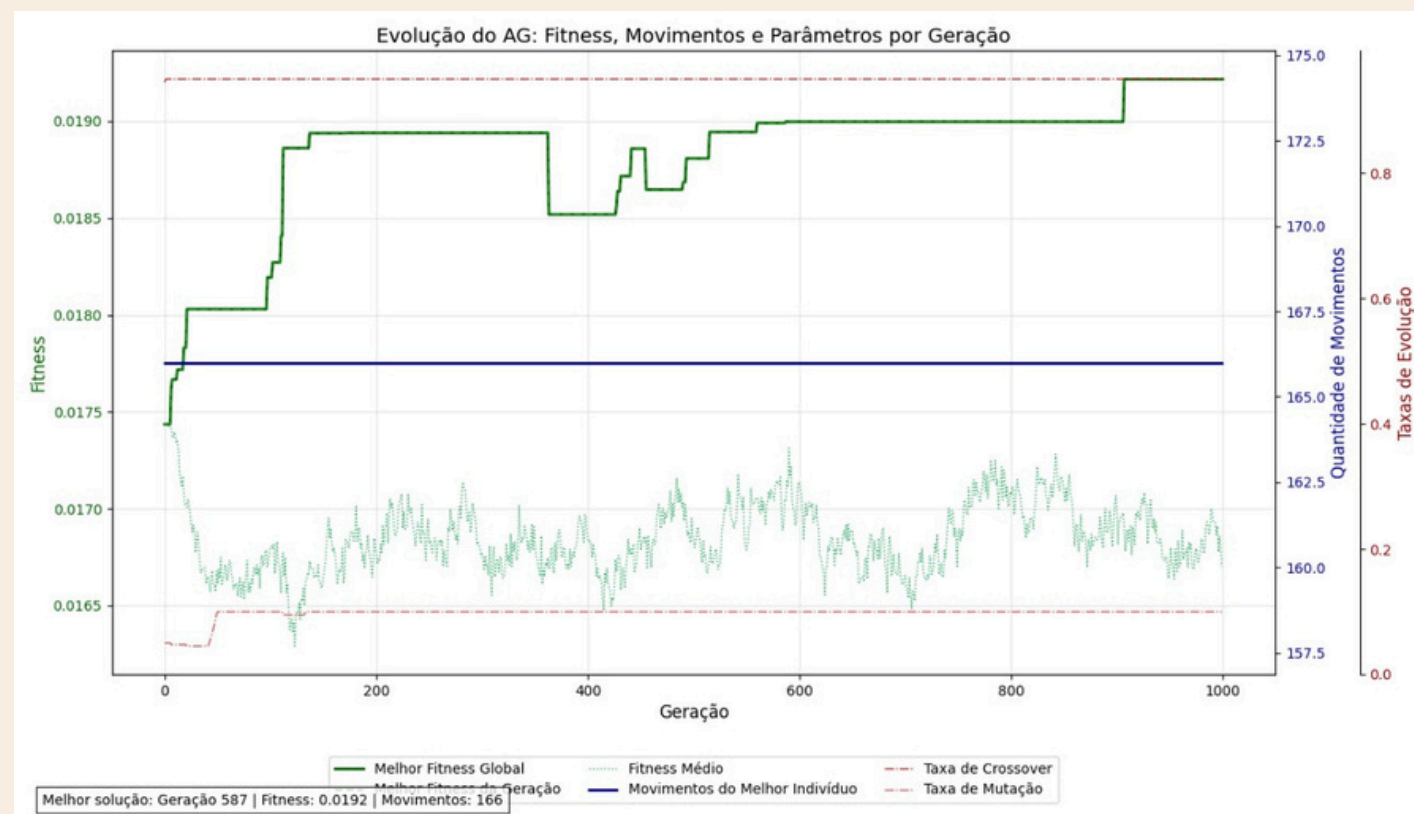
SOLUÇÃO ÓTIMA EM 2D



NOSSOS RESULTADOS

Equipe 2

EVOLUÇÃO DO AG



Resumo da Execução do Algoritmo Genético

- Evolução do Fitness
 - Geração 0: Fitness = 0.0178 (solução inicial aleatória)
 - Geração 300: Fitness = 0.0191 (primeira melhoria significativa)
 - Geração 999: Melhor fitness = 0.0195 (solução ótima encontrada)
- Características
 - Fitness: 0.0195 (quanto maior, melhor)
 - Movimentos Totais: 166 (abaixo do limite de 190)



MATERIAIS USADOS

- ✓ Isopor com medidas (70cm x 50cm)
- ✓ Potes de plástico como contêiner (200g)
- ✓ Areia, fita, papel filme, tinta para pintar barco
- ✓ Papel de seda para pintura de contêineres





CONCLUSÃO



- Resultados Alcançados
 - Balsa Estável
 - Distribuição otimizada pelo Algoritmo Genético.
 - Menor número de movimentos (166) e boa estabilidade.
 - Balsa Instável (Naufrágio)
 - Distribuição ruim proposital → Centro de massa desbalanceado.
 - Virou no protótipo físico, provando que:
 - Os contêineres têm peso suficiente para afetar a flutuação.
 - A organização espacial é crítica para a segurança.
- Animação feita no Blender
- Código feito em Python

